```
afterAfter((request, response) ->
response.header("Server", ""));
```

We can remove the leak of the exception class details by changing the exception handler to only return the error message not the full class, by changing the badRequest method you added earlier to only return the detail message from the exception.

```
private static void badRequest(Exception ex,
Request request, Response response) {
response.status(400);
response.body("{\"error\": \"" + ex.getMessage() + "\"}");
}
```

Cross-Site Scripting

Cross-site scripting, or XSS, is a common vulnerability affecting web applications, in which an attacker can cause a script to execute in the context of another site. In a *persistent* XSS, the script is stored in data on the server and then executed whenever a user accesses that data through the web application. A *reflected* XSS occurs when a maliciously crafted input to a request causes the script to be included (reflected) in the response to that request. Both can be devastating to the security of a web application, allowing an attacker to potentially steal session cookies and other credentials, and to read and alter data in that session. To appreciate why XSS is such a risk, you need to understand that the security model of web browsers is based on the *same-origin policy* (SOP). Scripts executing within the same origin (same site) as a web page are, by default, able to read cookies set by that website, examine HTML elements created by that site, make network requests to that site, and so on, although scripts from other origins are blocked from doing those things. A successful XSS allows an attacker to execute their script as if it came from a different origin, and so the malicious script gets to do all the same things that the genuine scripts from that origin can do. If I can successfully exploit an XSS vulnerability on facebook.com, for example, my script could potentially read and alter your Facebook posts.
Although XSS is primarily a vulnerability in web applications, in the age of single-page apps (SPAs) it’s common for web browser clients to talk directly to an API. For this reason, it’s